

TRABAJO FIN DE MÁSTER

Máster Big Data, Data Science & Inteligencia Artificial 2025-2026

Clasificación automática de tickets de soporte mediante NLP y Machine Learning: predicción de Type, Priority y Queue con un sistema de confianza AUTO / REVIEW / MANUAL

De la clasificación de tickets a una herramienta de soporte a la decisión: modelado, calibración de confianza y una aplicación Streamlit para la recogida de feedback

Autora: Judit Maymó

Tutores: Santiago Mota y Carlos Ortega

Fecha: septiembre de 2026

Contenido

1. Introducción	5
1.1. Contexto del proyecto	5
1.2. Problemática	5
1.3. Solución propuesta	6
1.4. Objetivos	6
1.5. Preguntas de investigación	7
2. Dataset	7
2.1. Descripción general	7
2.2. Variables	7
2.3. Análisis exploratorio de datos (EDA)	7
3. Preprocesamiento de los datos	9
4. Metodología de modelización	9
4.1. Representación del texto: TF-IDF	9
4.2. Algoritmos de clasificación	10
4.3. Métricas de evaluación	10
4.4. Estrategia experimental	11
5. Resultados del modelado con TF-IDF	11
5.1. Selección por algoritmo (Subject + Body)	11
5.2. Un cuarto candidato: Complement Naive Bayes	11
5.3. Incorporación de Tags	11
5.4. Evaluación en el conjunto de test	12
6. Comparación con representaciones basadas en embeddings	12
7. Análisis de los errores de clasificación	13
7.1. Pares de clases más confundidas	13
7.2. Patrones en los tickets mal clasificados	13
7.3. Coherencia entre Subject, Body y Tags	14
7.4. Síntesis del análisis de errores	15
8. Mejora del dataset y modelo final	15
8.1. Resultados en validación tras la fusión	15
8.2. Evaluación final en el conjunto de test	15
9. Calibración de la confianza y niveles AUTO / REVIEW / MANUAL	16
9.1. Comprobación de curvas de fiabilidad (confianza vs. accuracy real)	16
9.2. Definición de los umbrales AUTO / REVIEW / MANUAL	16
10. Aplicación Streamlit: sistema de soporte a la decisión	17
10.1. Roles y flujo de la aplicación	18
10.2. Persistencia y trazabilidad del feedback	18
10.3. Del feedback al reentrenamiento: el ciclo previsto	19
10.4. Propuesta: qué tickets conservar para el reentrenamiento	19
11. Discusión: ¿por qué las predicciones no son mejores?	19
12. Conclusiones y líneas futuras	20
12.1. Conclusiones	20
12.2. Limitaciones	20

12.3. Líneas futuras.....	20
Referencias	21
Anexos	22

1. Introducción

1.1. Contexto del proyecto

El presente Trabajo Fin de Máster se enmarca en un escenario simulado que sirve de hilo conductor para justificar tanto el desarrollo de los modelos de clasificación como el de la aplicación Streamlit que se presenta en el capítulo 10. Este contexto es ficticio, no corresponde a ninguna empresa real, pero se apoya en una situación habitual en organizaciones que operan con soporte técnico y volúmenes de tickets crecientes.

Escenario:

Una startup tecnológica ha ido acumulando un histórico de tickets de soporte —para este trabajo se utiliza como base un dataset real y disponible públicamente (descrito en el capítulo 2), que aquí se emplea como si fuera ese histórico— y desea automatizar la clasificación inicial de Type, Priority y Queue para agilizar la asignación de incidencias a sus equipos. La startup dispone de una cantidad de tickets etiquetados limitada y es consciente de que su calidad no es perfecta ya que el criterio de etiquetado no siempre ha sido consistente entre los distintos agentes que lo hicieron en su día.

Ante esta situación, el encargo que motiva este TFM tiene dos partes. La primera es puramente de modelado: entrenar, validar y comparar distintos algoritmos y representaciones de texto para predecir Type, Priority y Queue a partir de la información disponible cuando se crea un ticket (Subject, Body, Tags), incorporando una medida de confianza que permita decidir cuándo una predicción puede aceptarse automáticamente y cuándo requiere intervención humana.

La segunda parte es de producto: la startup necesita una aplicación que le permita empezar a operar con el modelo, aunque parta de la calidad de los datos actuales, y que a la vez sirva para seguir recogiendo tickets reales esta vez de calidad conocida. La idea central es que cada corrección que haga un técnico sobre una predicción se guarde junto con el ticket, de manera que, cuando se disponga de un volumen suficiente de nuevos tickets correctos o corregidos, se pueda reentrenar el modelo con ellos, de modo que en el futuro el modelo se preentrene con estos datos y sus predicciones merezcan una confianza cada vez más plena. El objetivo a medio plazo es que la proporción de predicciones correctas proporcionadas por el modelo crezca progresivamente a medida que el modelo aprende de datos propios, reales y revisados, en lugar de depender únicamente del dataset de partida.

Por tanto, este documento no presenta la aplicación Streamlit como el resultado de haber resuelto el problema de clasificación, sino como la pieza que permite convivir con un modelo imperfecto de forma segura. Para ello, la aplicación proporciona información sobre el nivel de confianza de las predicciones y reserva la decisión final a un técnico cuando sea necesario. Al mismo tiempo, establece las bases para la mejora continua del modelo a medida que se disponga de nuevos datos y experiencia.

1.2. Problemática

Se plantea el caso de una organización que necesita de un sistema de soporte para gestionar las consultas e incidencias comunicadas por sus usuarios. Cuando un usuario crea un ticket, describe por escrito su problema mediante un asunto (Subject), el contenido del mensaje (Body) y, opcionalmente, unas etiquetas (Tags). Antes de que el ticket pueda ser gestionado por un técnico, es necesario clasificarlo para determinar el tipo de incidencia (Type), la prioridad (Priority) y el departamento al que debe dirigirse (Queue).

La organización dispone de un histórico de tickets previamente gestionados en el que se conserva tanto la información proporcionada inicialmente por los usuarios como la clasificación asignada posteriormente. Este histórico constituye la base de datos sobre la que se desarrolla el presente trabajo, y permite estudiar si la información disponible en el momento de creación del ticket contiene patrones suficientes para automatizar esta clasificación inicial.

La clasificación de los tickets ya realizada puede haberse llevado a cabo de dos formas: por un lado, el propio usuario puede haber indicado los campos Type, Priority y Queue al crear el ticket, con el riesgo de que desconozca los criterios internos de la organización; por otro, un técnico puede haber realizado la clasificación

manualmente, evitando este problema, pero requiriendo su intervención en el 100 % de los tickets. Por ello, resulta de interés estudiar si la clasificación inicial puede automatizarse a partir de la información disponible, manteniendo la intervención de un técnico únicamente cuando la predicción no presente un nivel de fiabilidad suficiente.

1.3. Solución propuesta

Se propone desarrollar un sistema de clasificación automática basado en técnicas de aprendizaje automático y procesamiento de lenguaje natural (NLP), capaz de analizar la información disponible en el momento de creación de un ticket y predecir Type, Priority y Queue antes de que el ticket sea gestionado por un técnico.

El sistema incorpora una medida de confianza para cada una de las tres predicciones, que permite establecer, para cada una, uno de los siguientes niveles de actuación:

- **AUTO:** la confianza es suficientemente elevada y la predicción se acepta automáticamente; se muestra en verde.
- **REVIEW:** la confianza es intermedia; la predicción se presenta al técnico para su revisión, marcada en amarillo, lo que indica que deberá prestarle más atención antes de aceptarla.
- **MANUAL:** la confianza es demasiado baja; el técnico realiza directamente la clasificación, marcada en rojo, ya que lo más probable es que deba corregirla.

A partir de estas tres predicciones se determina el nivel de actuación del ticket en su conjunto, indicando qué variable o variables requieren intervención. Además, la intervención de los técnicos constituye una fuente de feedback: las predicciones revisadas o corregidas se guardan junto con la fecha de la corrección, lo que permite tanto identificar errores como estudiar, más adelante, la evolución de la calidad de las predicciones a lo largo del tiempo, y se conservan para su uso en futuras actualizaciones del modelo (capítulo 10).

1.4. Objetivos

Objetivo principal. Desarrollar y evaluar un sistema de NLP capaz de predecir automáticamente Type, Priority y Queue —tres objetivos de clasificación en total— a partir del asunto y la descripción textual de los tickets de soporte, incorporando un mecanismo de confianza que determine cuándo una clasificación puede automatizarse y cuándo requiere intervención humana. La pregunta que guía el proyecto es:

«¿Es posible construir un sistema de clasificación con una medida de confianza que permita automatizar una parte relevante de los tickets manteniendo un nivel de error aceptable?»

Esta pregunta traslada el problema desde una perspectiva puramente algorítmica —"¿qué accuracy tiene el modelo?"— hacia una perspectiva más próxima a su aplicación real: no basta con que el modelo acierte en promedio, sino que además debe ser capaz de reconocer cuándo puede confiarse en su propia predicción y cuándo no.

Objetivos específicos:

- Analizar las características del histórico de tickets y las categorías existentes de Type, Priority y Queue, así como su distribución y desbalance.
- Analizar las características lingüísticas de Subject y Body, y el valor de las etiquetas (Tags) como información adicional.
- Desarrollar y comparar distintos algoritmos y representaciones textuales (TF-IDF y embeddings) para las tres tareas de clasificación.
- Evaluar el rendimiento mediante métricas adecuadas a problemas desbalanceados, con F1 macro como criterio principal.
- Analizar si la confianza de las predicciones refleja razonablemente su probabilidad real de acierto (calibración), y entonces definir los umbrales AUTO / REVIEW / MANUAL.

- Identificar los principales errores del sistema y determinar si se deben a la dificultad del problema, a la calidad del etiquetado, o a ambas cosas.
- Diseñar un mecanismo de recogida de feedback de los técnicos, y una aplicación (Streamlit) que lo soporte de forma operativa.

1.5. Preguntas de investigación

- ¿Qué precisión puede alcanzarse en la predicción de Type, Priority y Queue a partir de Subject, Body y Tags? ¿Qué categorías son más fáciles y cuáles más difíciles?
- ¿La confianza proporcionada por el modelo es un indicador fiable de la corrección de sus predicciones? ¿Qué umbrales resultan razonables, a partir de ese diagnóstico, para separar los casos AUTO, REVIEW y MANUAL?
- ¿Es suficiente la información textual (Subject + Body), o las etiquetas (Tags) aportan información adicional relevante?
- ¿Qué tipos de errores comete el modelo y a qué se deben: solapamiento real entre clases, ruido en el etiquetado, o falta de información?
- ¿Puede el feedback de los técnicos utilizarse para construir un conjunto de datos de mayor calidad de cara a futuros reentrenamientos?

2. Dataset

2.1. Descripción general

El conjunto de datos utilizado corresponde a un dataset sintético de tickets de soporte multilingües, disponible públicamente en Kaggle. Representa un histórico de tickets en el que se conserva tanto la información proporcionada inicialmente por el usuario como distintos atributos de su clasificación y gestión posterior. Está compuesto por 28.587 registros y 16 variables, en inglés y alemán.

2.2. Variables

Variables de entrada. Subject (asunto) y Body (contenido del mensaje) son las principales fuentes textuales. El dataset contiene además ocho campos de etiquetas independientes (Tag_1–Tag_8) que en este trabajo se combinan y se tratan conceptualmente como una sola variable Tags. Language se usa para seleccionar los tickets en inglés, pero no como entrada del modelo.

Variables objetivo. Type (tipo de incidencia), Priority (prioridad) y Queue (departamento de destino). Cada una se trata como una tarea de clasificación independiente.

Variables excluidas. Answer, por generarse durante la gestión del ticket (no disponible en el momento de la clasificación inicial), y Version, sin información relevante para la tarea.

2.3. Análisis exploratorio de datos (EDA)

El análisis exploratorio se organiza en cuatro bloques: primero se revisa la estructura y calidad general del dataset (duplicados, valores ausentes, formato); a continuación se caracterizan Subject y Body como fuentes de texto; después se analizan las etiquetas (Tags) y su distribución de frecuencias; y, por último, se estudia la distribución y el desbalance de las tres variables objetivo (Type, Priority y Queue).

2.3.1. Estructura y calidad del dataset

No se identifican registros duplicados. Body, type, queue, priority, language y tag_1 no presentan valores ausentes; Subject sí presenta un porcentaje relevante de valores ausentes. Se detectan también variaciones de formato (mayúsculas/minúsculas, espacios) que se homogeneizarán en el preprocesamiento.

La distribución de Language muestra 16.338 tickets en inglés (57,15 %) y 12.249 en alemán (42,85 %). Al ser el inglés el idioma mayoritario y contar con volumen suficiente, el resto del trabajo se centra exclusivamente en los tickets en inglés, para trabajar con un conjunto lingüísticamente homogéneo.

2.3.2. Subject y Body

Subject presenta una mediana de 5 palabras y 42 caracteres; Body alcanza una mediana de 55 palabras y 377 caracteres, lo que confirma que Body concentra considerablemente más información textual. En cuanto a calidad del contenido, no se detectan URLs en ninguno de los dos campos; tampoco se detecta HTML en Subject, pero sí en 520 tickets de Body (3,18 %), que se elimina en el preprocesamiento por no aportar información relevante.

2.3.3. Las etiquetas (Tags)

Tras eliminar duplicados dentro de un mismo ticket, se identifican 886 etiquetas diferentes y 76.758 apariciones en los 16.338 tickets en inglés. La distribución de frecuencias es muy desigual: 400 etiquetas aparecen en un único ticket, mientras que solo 14 aparecen en más de 1.000. Esto motiva comparar distintos umbrales mínimos de frecuencia como variables de entrada.

Umbral mínimo	≥51	≥101	≥201	≥501	≥1.000
N.º de etiquetas distintas	86	52	33	22	14

Tabla 1. Número de etiquetas disponibles según el umbral mínimo de frecuencia (fuente: EDA, notebook).

En la configuración con el umbral más bajo (≥51), el 99,96 % de los tickets conservan al menos una etiqueta válida, con una media de 4,48 y una mediana de 4 etiquetas por ticket. Entre las más frecuentes destacan Tech Support, IT, Performance, Feedback, Bug, Security y Feature. Se decide no elegir un único umbral a priori, sino comparar los cinco candidatos (≥51, ≥101, ≥201, ≥501, ≥1.000) durante la fase de modelado mediante validación, evitando así una elección arbitraria.

2.3.4. Distribución de las variables objetivo

El Gráfico 1 muestra la distribución de categorías de Type, Priority y Queue, junto con su ratio de desbalance (frecuencia de la clase mayoritaria entre la minoritaria).

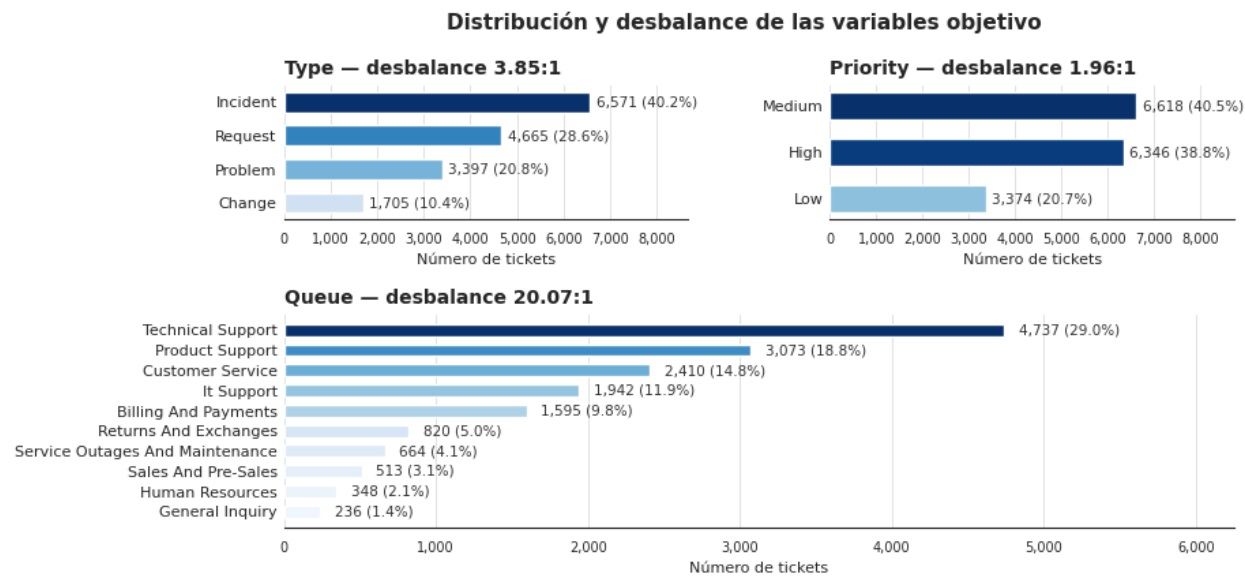


Gráfico 1. Distribución y desbalance de las variables objetivo.

Queue presenta el desbalance más acusado, lo que se tendrá en cuenta en la selección de métricas (F1 macro) y en el entrenamiento (class_weight="balanced").

3. Preprocesamiento de los datos

A partir de los resultados del EDA se aplican las siguientes transformaciones:

- Valores ausentes: se conservan. En Subject y Body, los valores ausentes se sustituyen por una cadena vacía en el momento de construir el texto de entrada a los modelos, de forma que el ticket conserva toda la información disponible en el resto de campos sin perder el registro.
- Homogeneización de formato: eliminación de espacios innecesarios y conversión a minúsculas en Subject, Body, Type, Priority, Queue, Language y Tags, para evitar que valores equivalentes se traten como distintos.
- Eliminación de contenido HTML: se limpian las etiquetas HTML presentes en el 3,18 % de los Body, conservando únicamente el texto útil.
- División de los datos: partición estratificada en entrenamiento (70 %), validación (15 %) y test (15 %), específica para cada variable objetivo, con semilla fija (`random_state=42`), de forma que se preserve la proporción de clases dado el desbalance detectado en el EDA (especialmente acusado en Queue). Cada partición resulta en 11.436 / 2.451 / 2.451 tickets, respectivamente, y se comprueba que las proporciones de clases se mantienen en las tres particiones.

La aplicación de otras técnicas lingüísticas (eliminación de stopwords, lematización) se decide y compara durante la fase de modelado, ya que forman parte de las propias configuraciones de representación de texto que se describen en el capítulo 4.

4. Metodología de modelización

4.1. Representación del texto: TF-IDF

Se opta por TF-IDF como primera representación textual, por ser una técnica adecuada para establecer una referencia sólida antes de considerar representaciones más costosas como los embeddings (capítulo 6). Sobre el texto ya normalizado en el capítulo 3, se aplica un paso de preparación específico de TF-IDF: la eliminación de stopwords (palabras muy frecuentes y con escaso valor discriminativo, como artículos o preposiciones), conservando explícitamente las negaciones (no, not, nor) por su posible relevancia semántica; se trata de un paso propio de esta representación, ya que los embeddings (capítulo 6) no aplican tratamiento de stopwords. De la lista estándar de 198 stopwords en inglés, tras excluir las tres negaciones quedan 195, de las cuales 118 aparecen realmente en Subject y Body.

Se generan dos versiones del corpus: sin lematizar y lematizada. La lematización consiste en reducir cada palabra a su forma canónica o lema (por ejemplo, "issues" e "issue" se agrupan bajo "issue"), lo que permite tratar como un único término distintas variantes de una misma palabra; en este corpus, reduce el vocabulario de 5.033 a 3.848 términos (−23,54 %). Se descarta el stemming por producir raíces que no siempre corresponden a palabras reales, y se descarta la representación basada en n-gramas de caracteres al no detectarse apenas errores ortográficos relevantes en el corpus.

Se establecen dos niveles de frecuencia mínima del vocabulario o `min_df` (el número mínimo de tickets en los que debe aparecer un término para conservarse: `min_df = 20` y `min_df = 100`) y dos rangos de n-gramas (solo unigramas, es decir, palabras sueltas, y unigramas + bigramas, es decir, también parejas de palabras consecutivas), sin seleccionar manualmente los términos por frecuencia para no descartar bigramas poco frecuentes pero potencialmente informativos para alguna clase. Combinando estas dos decisiones con la lematización se obtienen 8 configuraciones de representación TF-IDF:

Lematización	N-gramas	Frecuencia mínima
Sin lematizar	Unigramas	Baja (<code>min_df=20</code>)
Sin lematizar	Unigramas	Media (<code>min_df=100</code>)
Sin lematizar	Unigramas + bigramas	Baja (<code>min_df=20</code>)

Lematización	N-gramas	Frecuencia mínima
Sin lematizar	Unigramas + bigramas	Media (min_df=100)
Con lematización	Unigramas	Baja (min_df=20)
Con lematización	Unigramas	Media (min_df=100)
Con lematización	Unigramas + bigramas	Baja (min_df=20)
Con lematización	Unigramas + bigramas	Media (min_df=100)

Tabla 2. Ocho configuraciones de representación TF-IDF evaluadas.

4.2. Algoritmos de clasificación

Se seleccionan tres algoritmos lineales, adecuados para representaciones TF-IDF de alta dimensionalidad y dispersión, computacionalmente eficientes y utilizados habitualmente como referencia sólida en clasificación de texto: regresión logística, Linear SVM (LinearSVC) y SGDClassifier. Cada uno se evalúa con dos configuraciones de hiperparámetros (una más conservadora y otra más flexible en cuanto a regularización):

Algoritmo	Configuración 1 (conservadora)	Configuración 2 (flexible)
Regresión logística	C=1, class_weight="balanced", max_iter=1000	C=10, class_weight="balanced", max_iter=1000
Linear SVM (LinearSVC)	C=1, class_weight="balanced", max_iter=5000	C=10, class_weight="balanced", max_iter=5000
SGDClassifier	loss="hinge", alpha=0.0001, class_weight="balanced"	loss="hinge", alpha=0.001, class_weight="balanced"

Tabla 3. Algoritmos y configuraciones de hiperparámetros comparadas.

El parámetro C, en regresión logística y en LinearSVC, controla la regularización de forma inversa: valores bajos de C imponen una regularización más fuerte (modelo más simple, coeficientes más pequeños, menor riesgo de sobreajuste), mientras que valores altos permiten que el modelo se ajuste más a los datos de entrenamiento. En SGDClassifier, alpha cumple un papel equivalente pero en sentido directo: valores altos de alpha corresponden a mayor regularización. max_iter simplemente fija el número máximo de iteraciones del optimizador y se establece en un valor suficientemente alto (1.000 o 5.000, según el algoritmo) para asegurar la convergencia, sin que constituya en sí misma una decisión sobre la capacidad del modelo.

Se comparan dos valores de C (1 y 10) y de alpha (0,0001 y 0,001) que difieren en un orden de magnitud, para contrastar una regularización más fuerte frente a más débil, sin recurrir a una búsqueda exhaustiva de hiperparámetros: dado que cada configuración se combina con las 8 representaciones TF-IDF (16 combinaciones por algoritmo y variable, 189 en total), ampliar la rejilla habría multiplicado el coste computacional sin garantía de mejora sustancial sobre un problema ya dominado por la elección de representación textual y la calidad del etiquetado (capítulo 11).

class_weight="balanced" se utiliza en los tres algoritmos porque las variables objetivo presentan desbalance, especialmente Queue, de manera que las clases minoritarias reciben mayor peso durante el entrenamiento.

4.3. Métricas de evaluación

Se calculan accuracy, precision macro, recall macro y F1 macro. Aunque se consideran las cuatro para obtener una visión global, se utiliza F1 macro como criterio principal de comparación y selección, ya que otorga la misma importancia a todas las clases con independencia de su frecuencia. Esta elección es especialmente relevante para Queue, dado su desequilibrio, aunque también resulta adecuada para Type y Priority.

4.4. Estrategia experimental

El proceso se organiza en dos fases para cada variable objetivo (Type, Priority y Queue) y para cada uno de los tres algoritmos:

- Fase 1 — Subject + Body: se evalúan las 8 configuraciones TF-IDF combinadas con las 2 configuraciones de hiperparámetros del algoritmo (16 combinaciones), seleccionando la mejor según F1 macro en validación.
- Fase 2 — incorporación de Tags: manteniendo fija la mejor configuración de la fase 1, se añaden las etiquetas aplicando los cinco umbrales de frecuencia (51, 101, 201, 501, 1.000), seleccionando el umbral con mejor F1 macro en validación.

En total, cada algoritmo se evalúa mediante $16 + 5 = 21$ combinaciones por variable objetivo, es decir, 63 combinaciones por algoritmo y 189 combinaciones en el conjunto de las tres tareas. El conjunto de test permanece completamente al margen de esta comparación y solo se utiliza al final, sobre la configuración ya seleccionada.

5. Resultados del modelado con TF-IDF

5.1. Selección por algoritmo (Subject + Body)

Para cada variable objetivo se selecciona la configuración TF-IDF + hiperparámetros con mejor F1 macro en validación, dentro de cada algoritmo:

Objetivo	Regresión logística	Linear SVM	SGDClassifier
Type	0,8617	0,8646	0,8530
Priority	0,5904	0,5940	0,5733
Queue	0,5928	0,6082	0,4592

Tabla 4. F1 macro (validación) de la mejor configuración de cada algoritmo, por objetivo.

LinearSVC obtiene el mejor F1 macro en las tres variables objetivo, mientras que la regresión logística presenta resultados cercanos y SGDClassifier ofrece un rendimiento claramente inferior, especialmente en Queue. Por ello, SGDClassifier queda descartado como algoritmo candidato.

5.2. Un cuarto candidato: Complement Naive Bayes

Se añade Complement Naive Bayes (ComplementNB) como cuarto algoritmo, por ser una alternativa probabilística eficiente para matrices TF-IDF dispersas y potencialmente adecuada ante el desequilibrio de clases (se prueban $\alpha=0,1$ y $\alpha=1,0$). Random Forest se descarta de partida por su mayor coste computacional y menor adecuación a representaciones TF-IDF de alta dimensionalidad.

ComplementNB no mejora ningún resultado y su rendimiento es especialmente bajo en Queue, por lo que también queda descartado. Se mantiene, por tanto, LinearSVC como modelo principal para incorporar las etiquetas en la fase 2.

5.3. Incorporación de Tags

Partiendo de la mejor configuración TF-IDF de LinearSVC para cada objetivo, se comparan los cinco umbrales de frecuencia de Tags:

Objetivo	TF-IDF	Umbral de Tags	F1 macro (val.)
Type	con lematización, baja, bigramas, C=1	≥ 201	0,8691
Priority	sin lematización, baja, bigramas, C=1	≥ 201	0,6020

Objetivo	TF-IDF	Umbral de Tags	F1 macro (val.)
Queue	sin lematización, baja, bigramas, C=10	≥ 1.000	0,6251

Tabla 5. Mejor umbral de Tags por objetivo tras la incorporación al mejor LinearSVC.

Las etiquetas aportan una mejora moderada respecto al uso exclusivo de Subject y Body, confirmando su valor como información complementaria, aunque de magnitud distinta según el objetivo.

5.4. Evaluación en el conjunto de test

Objetivo	Accuracy	Precision macro	Recall macro	F1 macro
Type	0,8446	0,8584	0,8590	0,8584
Priority	0,6010	0,5866	0,5897	0,5879
Queue	0,5757	0,5842	0,5687	0,5756

Tabla 6. Resultados finales en test de TF-IDF + LinearSVC (antes de la mejora de datos del capítulo 8).

Objetivo	F1 validación	F1 test	Diferencia
Type	0,8691	0,8584	-0,0107
Priority	0,6020	0,5879	-0,0141
Queue	0,6251	0,5756	-0,0495

Tabla 7. Comparación entre validación y test: la caída es moderada en Type y Priority, y más apreciable en Queue.

El rendimiento de Queue no es necesariamente deficiente (F1 macro $\approx 0,576$ sobre 10 clases), pero la mayor caída entre validación y test sugiere que el modelo generaliza peor en esta variable, posiblemente por la combinación de más clases, mayor desbalance y mayor solapamiento conceptual entre categorías, cuestión que se analiza en profundidad en el capítulo 7.

6. Comparación con representaciones basadas en embeddings

Dadas las dificultades observadas en Priority y, sobre todo, en Queue —donde se confunden categorías conceptualmente relacionadas entre sí, como customer service, product support, technical support e IT support—, cabe preguntarse si una representación semántica densa (embeddings) distinguiría mejor estas categorías que TF-IDF, que solo detecta coincidencias literales de palabras y n-gramas y no captura relaciones de significado entre términos distintos.

Se generan embeddings con dos modelos de Sentence Transformers orientados al inglés: all-MiniLM-L6-v2, más ligero y de menor coste computacional, y all-mpnet-base-v2, de mayor capacidad semántica y coste más elevado. Sobre ambas representaciones se entrena el mismo clasificador (LinearSVC), con las mismas particiones train/validation/test y las mismas métricas, de forma que cualquier diferencia observada pueda atribuirse a la representación del texto.

Objetivo	TF-IDF + LinearSVC	Embeddings (MPNet) + LinearSVC	Diferencia
Type	0,8584	0,8139	-0,0445
Priority	0,5879	0,4663	-0,1216
Queue	0,5756	0,3879	-0,1877

Tabla 8. F1 macro en test: TF-IDF frente a embeddings de MPNet, con LinearSVC.

Contra lo que cabría esperar, TF-IDF supera claramente a los embeddings en las tres variables objetivo, y la diferencia es mayor precisamente en Queue, donde se esperaba que la representación semántica ayudase más. Esto sugiere que, en este dataset, la señal discriminativa reside en palabras y combinaciones de

términos concretas (vocabulario técnico específico, nombres de producto, expresiones de urgencia) más que en el significado general de la frase, y que además el volumen de datos (16.338 tickets en inglés) puede ser insuficiente para que una representación semántica genérica, no ajustada (fine-tuned) a este dominio, aporte una ventaja frente a TF-IDF.

Por tanto, se selecciona TF-IDF + LinearSVC como combinación final para las tres tareas de clasificación, con una configuración específica por objetivo (para Type, con lematización, unigramas y bigramas, min_df=20, C=1; para Priority y Queue, sin lematización, unigramas y bigramas, min_df=20, con C=1 y C=10, respectivamente).

7. Análisis de los errores de clasificación

Una vez seleccionada la combinación TF-IDF + LinearSVC, el objetivo de este capítulo es entender el origen de los errores antes de decidir cómo actuar sobre ellos. Concretamente, se busca distinguir entre tres posibles causas: problemas de calidad de los datos que serían corregibles, información insuficiente o ambigua en los campos disponibles, y solapamiento real entre categorías, que constituye una limitación del propio dataset y no del modelo. Para ello se combinan tres tipos de análisis: el estudio de las clases que más se confunden entre sí, la búsqueda de patrones en los tickets mal clasificados, y la comparación de la información aportada de forma independiente por Subject, Body y Tags.

7.1. Pares de clases más confundidas

Las matrices de confusión completas (test) se incluyen en el Anexo [2]. Los pares con mayor número de errores son:

- **Type: *incident y problem***: comparten casi todo el vocabulario (issue, problem, system, resolve, security). 205 tickets de incident se clasifican como problem y 168 a la inversa, lo que explica su F1 más bajo (0,806 y 0,646) frente a change y request (0,986 y 0,996). Se fusionan ambas clases (capítulo 8).
- **Priority: *high, medium y low***: high presenta señales léxicas claras de urgencia; medium y low apenas se distinguen entre sí. Se detectan tickets graves (brechas de seguridad, accesos a datos médicos) etiquetados como medium o low, lo que apunta a ruido de etiquetado histórico más que a dificultad lingüística. Se mantienen las tres clases al no disponer de una fuente externa para corregir el etiquetado.
- **Queue: *technical support, product support, IT support y customer service***: existe solapamiento léxico fuerte entre las tres primeras (issue, software, system, security, integration). Se fusionan technical support y product support por presentar la frontera más débil y sin vocabulario claramente diferenciador; IT support se mantiene separada porque conserva señal propia (herramientas, conectividad) suficiente para distinguirla. Customer service se mantiene igualmente separada: su confusión con las categorías técnicas no parece deberse a solapamiento conceptual sino a inconsistencias del etiquetado histórico (tickets de contenido técnico etiquetados como customer service), y fusionarla ocultaría ese problema de calidad de datos en lugar de resolver una ambigüedad real, además de eliminar una distinción con valor operativo claro para la organización.

7.2. Patrones en los tickets mal clasificados

Longitud del ticket. No se encuentra evidencia de que los errores se concentren en tickets especialmente cortos o largos: las medianas de longitud (Subject, Body, Tags) son prácticamente iguales entre aciertos y errores en las tres variables.

Disponibilidad de información. Body y Tags están disponibles en el 100 % de los casos, tanto en aciertos como en errores. Sorprendentemente, los tickets mal clasificados presentan incluso una mayor proporción de Subject disponible que los correctamente clasificados (por ejemplo, en Type, 88,98 % frente a 82,46 %). Por

tanto, la ausencia de información no explica los errores: el problema no es que falten datos, sino que el contenido disponible resulta difícil de interpretar o distinguir entre clases próximas.

7.3. Coherencia entre Subject, Body y Tags

El análisis más revelador consiste en comprobar si, en los tickets mal clasificados, Subject, Body y Tags apuntan hacia categorías distintas. Para ello se entrena, para cada objetivo, un LinearSVC independiente sobre cada campo por separado (usando la configuración TF-IDF ya seleccionada), y se comparan las tres predicciones resultantes para cada ticket.

Objetivo	Body (F1 macro)	Subject (F1 macro)	Tags (F1 macro)
Type	0,8590	0,6272	0,6053
Priority	0,5592	0,3968	0,3893
Queue	0,5495	0,2435	0,2415

Tabla 9. Rendimiento de cada campo por separado (validación). Body concentra la mayor parte de la señal predictiva.

Dado que Subject y Tags por separado tienen muy poca señal individual (especialmente en Queue), no tendría sentido usarlos de forma aislada como clasificadores; el objetivo de este análisis es detectar incoherencia entre fuentes de información, no evaluar cada campo como clasificador independiente. Por ello, se compara Body frente a la combinación Subject + Tags (con el umbral de Tags ≥ 201), y se define una variable de coherencia (True/False) según si ambas fuentes coinciden en su predicción sobre el conjunto de validación.

Objetivo	Coherencia entre Body y Subject+Tags
Type	70,95 %
Priority	46,47 %
Queue	28,93 %

Tabla 10. Porcentaje de tickets en los que Body y Subject+Tags predicen la misma clase.

A continuación se cruza esta variable de coherencia con el acierto/error del modelo principal (LinearSVC entrenado con Subject+Body+Tags):

Objetivo	Coherencia	Total	Aciertos	Errores	% acierto	% error
Type	No	712	529	183	74,30	25,70
Type	Sí	1.739	1.576	163	90,63	9,37
Priority	No	1.312	773	539	58,92	41,08
Priority	Sí	1.139	737	402	64,71	35,29
Queue	No	1.742	954	788	54,76	45,24
Queue	Sí	709	523	186	73,77	26,23

Tabla 11. Tasa de error del modelo principal según haya (Sí) o no (No) coherencia entre Body y Subject+Tags.

La relación se observa claramente en los tres casos, aunque con distinta magnitud. En Type, la tasa de error se reduce del 25,70 % al 9,37 % cuando ambas fuentes coinciden. En Queue, el efecto es aún más pronunciado, pasando del 45,24 % al 26,23 %. Por su parte, en Priority la reducción es más moderada, del 41,08 % al 35,29 %, lo que resulta coherente con la hipótesis de que su principal dificultad no reside tanto en la falta de coherencia entre campos como en el ruido presente en el etiquetado. Esto confirma que, cuando los distintos campos de un mismo ticket aportan señales contradictorias entre sí, el modelo tiene más dificultades, pero también apunta a cierta falta de coherencia en la calidad de la base de datos, ya que no necesariamente implica que esos tickets sean erróneos: pueden ser tickets legítimos en los que distintos campos aportan información complementaria pero no siempre consistente.

7.4. Síntesis del análisis de errores

- **Type:** Se identifica un solapamiento léxico prácticamente total entre las clases `incident` y `problem`, sin señales diferenciadoras suficientes que permitan garantizar una separación fiable. En consecuencia, se decide fusionar ambas categorías en una única clase, denominada `incident_problem`, tal como se detalla en el capítulo 8.
- **Priority:** El principal problema identificado es la presencia de ruido e inconsistencias en el etiquetado histórico, especialmente en tickets de elevada gravedad clasificados como `medium` o `low`. Dado que no se dispone de una fuente externa que permita validar o corregir estas etiquetas de forma objetiva, se decide mantener las tres clases originales, documentar explícitamente esta limitación y evitar tanto la fusión de categorías como la corrección manual del etiquetado.
- **Queue:** Se observa un solapamiento conceptual elevado entre las clases `technical support` y `product support`, así como un solapamiento de menor magnitud con `IT support`. Además, la clase `customer service` presenta indicios de inconsistencia en su etiquetado. Por ello, se decide fusionar `technical support` y `product support`, manteniendo `IT support` y `customer service` como clases independientes, tal como se describe en el capítulo 8.

8. Mejora del dataset y modelo final

A partir de las conclusiones del capítulo 7, se aplican dos fusiones de categorías directamente sobre las variables objetivo, y se reentrena el mejor pipeline (TF-IDF + LinearSVC) manteniendo la misma configuración de representación e hiperparámetros ya seleccionada para cada objetivo, pero recalculando las particiones `train/validation/test` (necesario porque las etiquetas de `Type` y `Queue` han cambiado).

- **Type:** se fusionan `incident` y `problem` en una única categoría, `incident_problem`.
- **Queue:** se fusionan `technical support` y `product support` en una única categoría, `technical_product_support`. Se mantienen separadas `IT support` y `customer service`.

Tras la fusión, `incident_problem` representa el 61,01 % de `Type` y `technical_product_support` el 47,80 % de `Queue`: el desbalance no desaparece, simplemente se reorganiza, por lo que se mantiene `class_weight="balanced"` en el entrenamiento. `Priority` no se modifica.

8.1. Resultados en validación tras la fusión

Objetivo	F1 macro antes de fusionar	F1 macro después de fusionar
Type	0,8691	0,9945
Priority	0,6020	0,6020 (sin cambios)
Queue	0,6251	0,6346

Tabla 12. Efecto de la fusión de categorías sobre el F1 macro en validación.

En `Type`, la mejora es drástica: al eliminar la principal fuente de confusión (`incident` vs. `problem`), el F1 macro pasa de 0,8691 a 0,9945. En `Queue`, la mejora es más moderada (de 0,6251 a 0,6346), porque la fusión resuelve una fuente de error pero persisten otras categorías con vocabulario parecido (`IT support`, `customer service`). `Priority` permanece prácticamente sin cambios, lo cual es coherente con el diagnóstico del capítulo 7: su problema no era un solapamiento entre dos categorías concretas, sino ruido de etiquetado distribuido de forma más general.

8.2. Evaluación final en el conjunto de test

Objetivo	Accuracy	Precision macro	Recall macro	F1 macro
Type	0,9959	0,9941	0,9940	0,9941
Priority	0,6010	0,5866	0,5897	0,5879

Objetivo	Accuracy	Precision macro	Recall macro	F1 macro
Queue	0,6430	0,6145	0,5856	0,5970

Tabla 13. Resultados finales en test del modelo mejorado (TF-IDF + LinearSVC, con fusión de categorías).

Estos son los resultados que finalmente se integran en la aplicación Streamlit (capítulo 10). La mejora de datos funciona de forma muy clara para Type (F1 macro test $\approx 0,994$), mejora moderadamente Queue (de 0,5756 a 0,5970) y no afecta a Priority (0,5879), que se mantiene como la variable con menor margen de mejora dentro de este enfoque basado en fusión de categorías.

9. Calibración de la confianza y niveles AUTO / REVIEW / MANUAL

Un LinearSVC no calcula probabilidades por sí mismo, solo devuelve mediante `decision_function`, la distancia de cada ticket a la frontera entre clases, un número sin escala de probabilidad definida. Calibrar el modelo consiste en ajustar esas puntuaciones para que sí reflejen la probabilidad real de acierto (que un ticket con confianza 0,80 acierte realmente en torno al 80 % de las veces), algo necesario antes de fijar los umbrales AUTO/REVIEW/MANUAL.

Se utiliza `CalibratedClassifierCV` con `method="sigmoid"` (calibración de Platt), ajustada sobre el conjunto de validación: al ajustar una única curva sigmoide con muy pocos parámetros, funciona bien incluso con clases con pocos ejemplos, como la minoritaria de Queue. El clasificador de base no se reentrena: solo se ajusta la función que traduce su puntuación en una probabilidad.

9.1. Comprobación de curvas de fiabilidad (confianza vs. accuracy real)

Calibrar el modelo no garantiza que la calibración haya funcionado bien. Por este motivo hay que comprobarlo: se ordenan los tickets de test de mayor a menor confianza y se dividen en 10 grupos de tamaño igual (el primero, con el 10 % de tickets en los que el modelo está más seguro; el último, con el 10 % en los que está menos seguro). Dentro de cada grupo se calcula el accuracy real, es decir, qué porcentaje de esos tickets el modelo clasificó realmente bien. Si la calibración es buena, el accuracy real debería ser alto en los grupos de mayor confianza y bajar progresivamente hacia los de menor confianza. La Tabla 16 muestra, para cada variable, el grupo de mayor y el de menor confianza.

Objetivo	Confianza (bin más alto)	Accuracy real	Confianza (bin más bajo)	Accuracy real
Type	0,990 – 1,000	$\approx 1,000$	0,473 – 0,990	0,955
Priority	0,840 – 0,979	0,939	0,226 – 0,412	0,367
Queue	0,741 – 0,929	0,862	0,431 – 0,468	0,461

Tabla 14. Curvas de fiabilidad (test): accuracy real en el bin de mayor y de menor confianza, por variable.

En Type, incluso el grupo de menor confianza acierta el 95,5% de las veces: el modelo es fiable en prácticamente todo el rango. En Priority ni siquiera el grupo de mayor confianza supera el 90 %, y en Queue se queda justo por debajo del 95%. Esto confirma que la confianza calibrada sí ordena correctamente los tickets de más a menos fiables, pero también que, en Priority y Queue, incluso las predicciones "más seguras" del modelo no son tan fiables como en Type.

9.2. Definición de los umbrales AUTO / REVIEW / MANUAL

Se fijan dos umbrales globales, iguales para las tres variables: confianza $\geq 0,80$ corresponde a AUTO, entre 0,60 y 0,80 a REVIEW, y por debajo de 0,60 a MANUAL. Se trata de una elección de partida razonable, fácil de comunicar y coherente con el diagnóstico anterior: 0,80 corresponde en Type a una zona de accuracy casi perfecta, y en Priority y Queue al primer o segundo bin de la curva de fiabilidad, donde el acierto real todavía es razonablemente alto. Queda pendiente, como línea de mejora futura (capítulo 12), optimizar estos umbrales de forma específica por variable según el coste real de cada tipo de error..

Nivel	Rango de confianza	Acción en la aplicación
AUTO (verde)	$\geq 0,80$	La predicción se acepta automáticamente; se muestra en verde.
REVIEW (amarillo)	0,60 – 0,80	La predicción se muestra al técnico para su revisión antes de aceptarla; se muestra en amarillo.
MANUAL (rojo)	$< 0,60$	El técnico debe clasificar directamente esa variable; se muestra en rojo.

Tabla 15. Niveles de actuación y su correspondencia visual en la aplicación.

Dado el comportamiento observado en las curvas de fiabilidad, es esperable que, con el modelo actual, una parte relevante de los tickets aparezca en amarillo o rojo en Priority y Queue, y en cambio la gran mayoría de los tickets aparezca en verde en Type. Esto se retoma explícitamente en la discusión del capítulo 11.

Objetivo	% AUTO	% REVIEW	% MANUAL
Type	99,22	0,61	0,16
Priority	4,08	36,39	59,53
Queue	16,48	35,58	47,94

Tabla 16. Porcentaje de tickets de test en cada nivel de actuación, por variable.

Aplicando estos umbrales sobre el conjunto de test, la distribución de tickets por nivel de actuación confirma de forma cuantitativa lo anticipado: en Type, el 99,22 % de los tickets cae en AUTO, frente a solo el 4,08 % en Priority y el 16,48 % en Queue, donde la mayoría de tickets requeriría REVIEW o MANUAL.

Objetivo	Nivel	Total	Aciertos	% aciertos
Type	AUTO	2.432	2.427	99,79
Type	REVIEW	15	13	86,67
Type	MANUAL	4	0	0,00
Priority	AUTO	100	89	89,00
Priority	REVIEW	892	648	72,65
Priority	MANUAL	1.459	750	51,41
Queue	AUTO	404	371	91,83
Queue	REVIEW	872	653	74,89
Queue	MANUAL	1.175	597	50,81

Tabla 17. Acierto real dentro de cada nivel de actuación, por variable.

Esta tabla confirma que el sistema de niveles funciona como se pretende: en las tres variables, el acierto real cae de forma monótona de AUTO a MANUAL, validando que la confianza calibrada ordena correctamente la fiabilidad de las predicciones. El único caso llamativo es Type MANUAL, con 0 % de acierto sobre solo 4 tickets — los residuos del solapamiento entre incident y problem que la fusión de categorías del capítulo 8 no llegó a resolver del todo.

10. Aplicación Streamlit: sistema de soporte a la decisión

La aplicación Streamlit implementa, como herramienta operativa, el sistema descrito en los capítulos anteriores: aplica el modelo calibrado (capítulo 9) sobre cada ticket, muestra su nivel de confianza mediante el código de colores AUTO / REVIEW / MANUAL, y registra de forma sistemática las correcciones de los técnicos para su uso en futuros reentrenamientos. Ver Aplicación en Anexo [3].

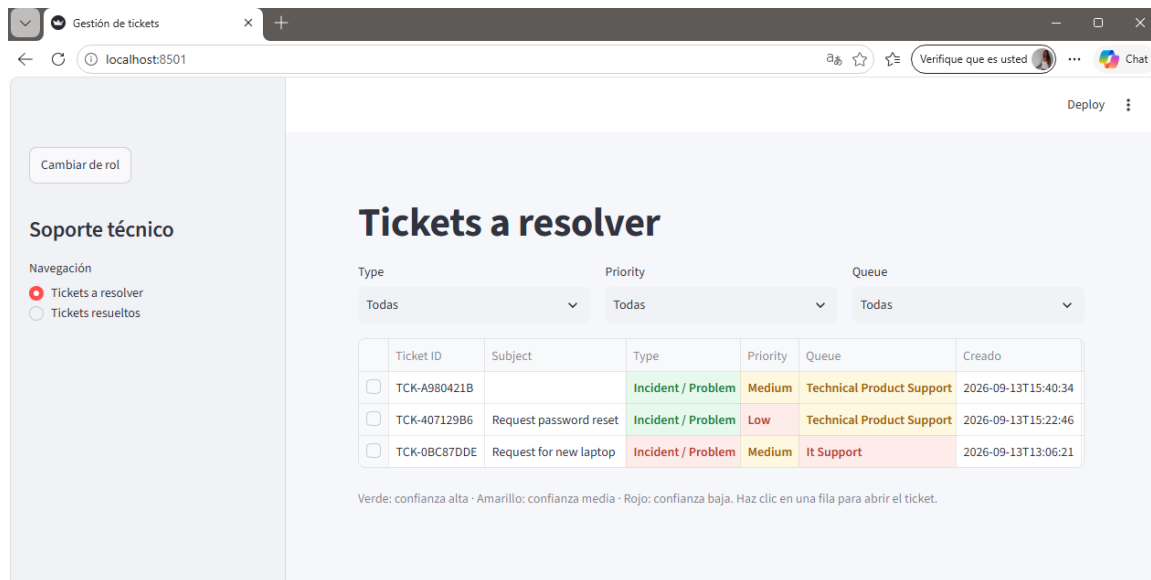


Figura 1.

10.1. Roles y flujo de la aplicación

Existen dos formas de acceder a la aplicación: como Usuario o como Técnico de soporte. El Usuario únicamente crea tickets; el Técnico dispone de dos vistas, Tickets a resolver y Tickets resueltos.

Usuario. Crea un ticket (en inglés) indicando Subject, Body y, opcionalmente, Tags, y recibe la confirmación de que se ha registrado correctamente. No ve ninguna predicción del modelo, lo que mantiene el foco de la clasificación en el equipo técnico.

Técnico — Tickets a resolver. Se muestran todos los tickets pendientes en una tabla con filtros por Type, Priority y Queue. Las columnas predichas se colorean en verde / amarillo / rojo según el nivel de confianza calibrado (capítulo 9). Al hacer clic en una fila se abre el detalle del ticket, y solo entonces se muestra el porcentaje de confianza exacto. Desde el detalle, el técnico puede corregir Type, Priority y/o Queue (o dejarlos igual si está de acuerdo con la predicción) y marcar el ticket como resuelto.

Técnico — Tickets resueltos. Listado sin colores de confianza (ya no aportan valor una vez resuelto el ticket), que indica si la predicción automática fue modificada por el técnico o no. Al abrir el detalle se resume de forma concisa qué cambió respecto a la predicción original.

10.2. Persistencia y trazabilidad del feedback

La aplicación utiliza una base de datos SQLite con tres tablas:

- tickets_totales: como registro de auditoría con todos los tickets creados.
- tickets_pendientes: tickets aún sin resolver.
- tickets_resueltos: además de la clasificación final (tras la posible corrección del técnico), guarda también la predicción automática original (pred*_original), lo que permite calcular en cualquier momento si un ticket fue modificado y, en caso afirmativo, qué campo concreto cambió (Type, Priority y/o Queue) y desde qué valor hasta cuál.

Esta trazabilidad es lo que convierte a la aplicación en algo más que una interfaz de predicción: cada ticket resuelto queda etiquetado, de forma implícita, con una "etiqueta de confianza en la corrección" (si el técnico la tocó o no), lo que en el futuro permitirá filtrar y priorizar qué tickets usar para reentrenar.

10.3. Del feedback al reentrenamiento: el ciclo previsto

El ciclo de mejora continua, donde los técnicos corrigen, las correcciones se almacenan, y con volumen suficiente el modelo se reentrena siguiendo como partida la misma metodología de este trabajo (capítulos 4 a 8), es, en el estado actual, un proceso manual y offline: la aplicación recoge y estructura el feedback, pero no ejecuta el reentrenamiento ni lo valida automáticamente. Automatizar ese ciclo queda como línea de trabajo futura (capítulo 12): esto incluiría tanto el reentrenamiento programado como una validación automática que confirme que el nuevo modelo no empeora al anterior antes de sustituirlo en producción.

10.4. Propuesta: qué tickets conservar para el reentrenamiento

La aplicación guarda todos los tickets resueltos, con o sin corrección del técnico. Se propone que, de cara al reentrenamiento futuro del modelo, puedan considerarse dos alternativas: usar todos los tickets guardados, o utilizar únicamente los que fueron corregidos (originalmente en amarillo o rojo), por ser los que realmente aportan información nueva al modelo. Guardar también los aciertos en verde (AUTO) tiene valor igualmente, ya que permite confirmar que el modelo sigue funcionando bien en esa zona de confianza a medida que llegan datos reales.

11. Discusión: ¿por qué las predicciones no son mejores?

Como rendimiento obtenido, un F1 macro en test de 0,994 para Type es excelente, pero 0,588 en Priority y 0,597 en Queue distan de resultados con los que automatizar con tranquilidad la mayoría de los tickets. A partir de la evidencia de los capítulos 6 a 9, se identifican tres factores, no excluyentes entre sí:

11.1. Volumen de datos limitado

El entrenamiento se realiza sobre 11.436 tickets, un volumen modesto para un problema con hasta 10 clases (Queue). Es coherente con el peor comportamiento de los embeddings frente a TF-IDF visto en el capítulo 6: una representación semántica genérica y no ajustada al dominio necesita más datos para superar a una representación léxica simple. Con más tickets reales, representaciones más ricas podrían empezar a aportar la ventaja que aquí no se observa.

11.2. Calidad y consistencia del etiquetado

El análisis de errores del capítulo 7 apunta a que parte del problema es de calidad del etiquetado histórico: tickets graves etiquetados como medium o low en Priority, o tickets técnicos etiquetados como customer service en Queue. Ningún modelo puede aprender de forma fiable una distinción que no está reflejada de forma coherente en los datos de entrenamiento.

11.3. Solapamiento conceptual real entre categorías

Existe además solapamiento genuino entre categorías que tratan problemas parecidos (incident/problem, technical support/product support/IT support). La fusión de categorías del capítulo 8 lo aborda parcialmente, con una mejora muy notable en Type pero solo moderada en Queue, porque persisten fronteras débiles con IT support y customer service: no todo el solapamiento puede resolverse fusionando categorías sin perder la granularidad que necesita la organización.

En conjunto, estas limitaciones no responden a una mala elección de algoritmo o representación — ampliamente comparados en este trabajo —, sino a la combinación de datos limitados y, sobre todo, etiquetado histórico poco consistente. Por este motivo la mejor opción para mejorar Priority y Queue es recoger y reentrenar con nuevos tickets correctos a través de la aplicación.

12. Conclusiones y líneas futuras

12.1. Conclusiones

El proyecto demuestra que la clasificación automática de tickets es viable, pero de forma desigual según qué se prediga. Type se puede automatizar con confianza: más de 9 de cada 10 tickets se clasificarán en verde (AUTO) con un acierto cercano al 100 %, gracias a fusionar dos categorías que en la práctica describían lo mismo (incident/problem). Priority y Queue no están listas para automatizarse sin supervisión: incluso en sus predicciones más seguras, el modelo falla entre 1 de cada 7 y 1 de cada 16 veces. La causa principal no es una mala elección de algoritmo —se compararon 4 algoritmos y 2 representaciones de texto (TF-IDF y embeddings), evaluando 189 configuraciones en total durante la fase de TF-IDF (63 por variable objetivo)— sino la calidad del histórico: tickets graves etiquetados como "low", o de soporte técnico etiquetados como "customer service", que ningún modelo puede aprender a corregir si el propio dato de entrenamiento es inconsistente.

Por eso el sistema de niveles AUTO / REVIEW / MANUAL no es una capa opcional sobre el modelo, es la pieza que lo hace utilizable hoy: permite beneficiarse de la parte que ya funciona (Type) sin arriesgarse a aceptar automáticamente errores en las partes que aún no funcionan con seguridad (Priority, Queue). Y la aplicación Streamlit convierte cada corrección de un técnico en un dato de entrenamiento futuro, de forma que la calidad del modelo pueda mejorar con el tiempo sin depender de un dataset inicial que se sabe imperfecto.

La mejora concreta y medible aportada por este trabajo es doble: pasar de un F1 macro de 0,86 a 0,99 en Type solo con una decisión de negocio (fusionar dos categorías que generaban confusión), y convertir un "el modelo acierta el 60 % de las veces" en algo accionable — "el modelo acierta el 94 % de las veces cuando dice estar seguro, y avisa cuando no lo está" — que es la diferencia entre un modelo no útil en producción y uno que sí puede desplegarse de forma responsable.

12.2. Limitaciones

- El dataset se seleccionó por incluir simultáneamente las tres variables objetivo (Type, Priority, Queue) y un volumen suficiente de registros, algo poco frecuente en datasets públicos de este tipo. Su origen es sintético, lo cual no es la opción recomendada para un TFM y puede limitar el alcance de las conclusiones. En particular, el análisis de «inconsistencia de etiquetado» de los capítulos 7 y 11 describe un patrón observado en estos datos concretos, que un futuro reentrenamiento con tickets reales (capítulo 10) debería confirmar.
- Solo se ha trabajado con la parte en inglés del dataset (16.338 de 28.587 tickets), lo que reduce aún más el volumen disponible para entrenamiento y validación.
- Los umbrales AUTO/REVIEW/MANUAL (0,80 / 0,60) son globales para las tres variables; no se ha optimizado el coste-beneficio por variable ni por tipo de error.
- El reentrenamiento con el feedback de los técnicos no está automatizado; es, por ahora, un proceso manual y offline.
- La fusión de categorías en Type y Queue implica una pérdida de granularidad que debería validarse con la organización antes de adoptarse en un entorno real.

12.3. Líneas futuras

- Recoger tickets reales a través de la aplicación, estudiar y reentrenar periódicamente el modelo, comparando su rendimiento con el actual antes de sustituirlo en producción.
- Revisar con la organización el criterio de etiquetado de Priority para reducir el ruido detectado en el capítulo 7.
- Optimizar los umbrales AUTO/REVIEW/MANUAL por variable, e incluso por clase, según el coste real de cada tipo de error.

- Evaluar el ajuste fino (fine-tuning) de un modelo tipo BERT/Sentence-Transformer sobre este dominio, una vez se disponga de más datos reales.
- Incorporar aprendizaje activo, priorizando para revisión humana los tickets en la zona REVIEW donde el modelo tiene más que aprender.
- Automatizar el ciclo de reentrenamiento y despliegue, con validación automática de que el nuevo modelo no empeora al anterior.

Referencias

- Kaggle. (s. f.). *Multilingual Customer Support Tickets* [Conjunto de datos]. Kaggle. <https://www.kaggle.com/datasets/tobiasbueck/multilingual-customer-support-tickets>
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. En A. J. Smola, P. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), *Advances in large margin classifiers* (pp. 61–74). MIT Press.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 3982–3992.
- Rennie, J. D., Shih, L., Teevan, J., & Karger, D. R. (2003). Tackling the poor assumptions of naive Bayes text classifiers. *Proceedings of the 20th International Conference on Machine Learning (ICML-2003)*, 616–623.
- Scikit-learn. (s. f.). *Scikit-learn documentation*. <https://scikit-learn.org/>
- Streamlit. (s. f.). *Streamlit documentation*. <https://docs.streamlit.io/>

Anexos

- Anexo 1: Código fuente y repositorio:

El código fuente desarrollado para la realización de este proyecto se encuentra disponible en un repositorio de GitHub, junto con los archivos y recursos necesarios para la ejecución y reproducción del proyecto.

Repositorio completo de GitHub:

<https://github.com/JuditMaymo/TFM>

Código fuente principal:

<https://nbviewer.org/github/JuditMaymo/TFM/blob/main/TFM.ipynb>

- Anexo 2: Matrices de confusión de las variables objetivo

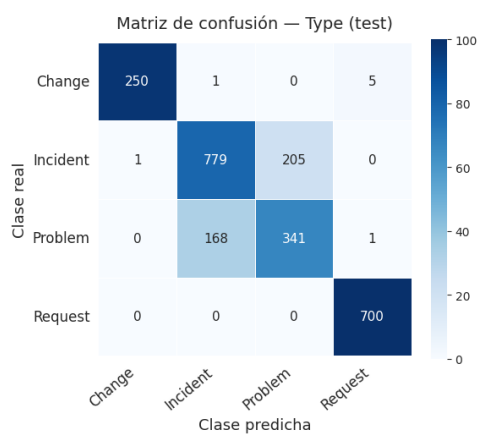


Figura A2.1

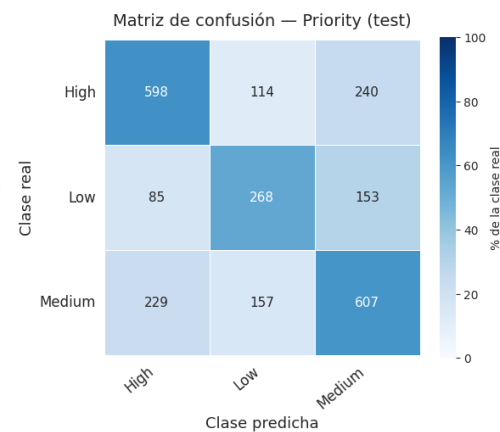


Figura A2.2

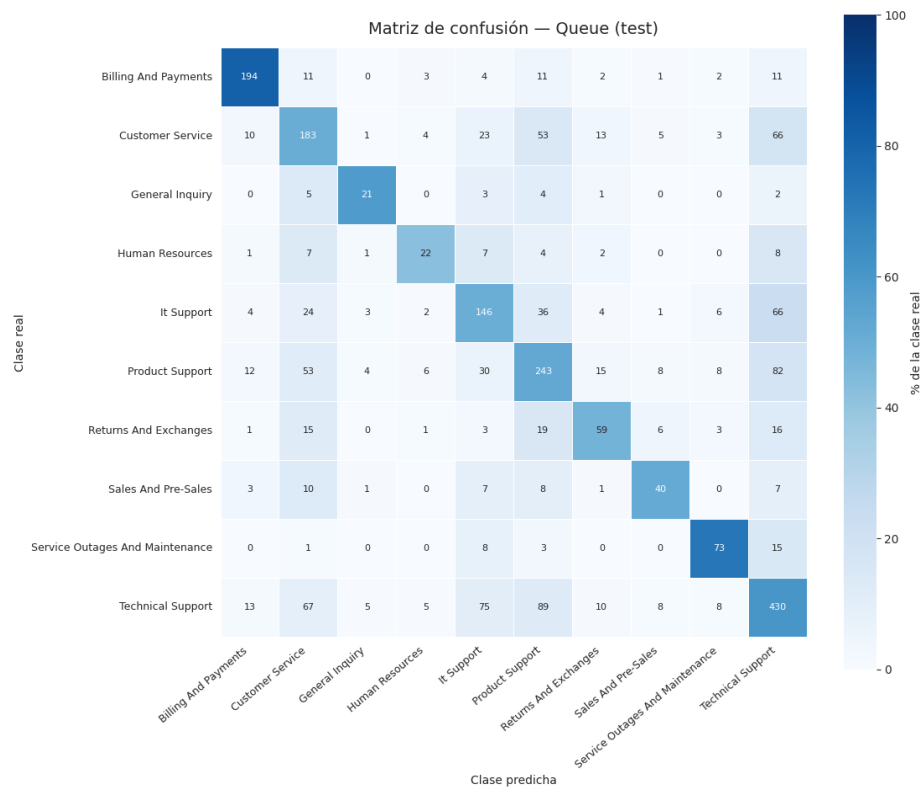


Figura A2.3

- Anexo 3: Aplicación Streamlit

Pantalla principal

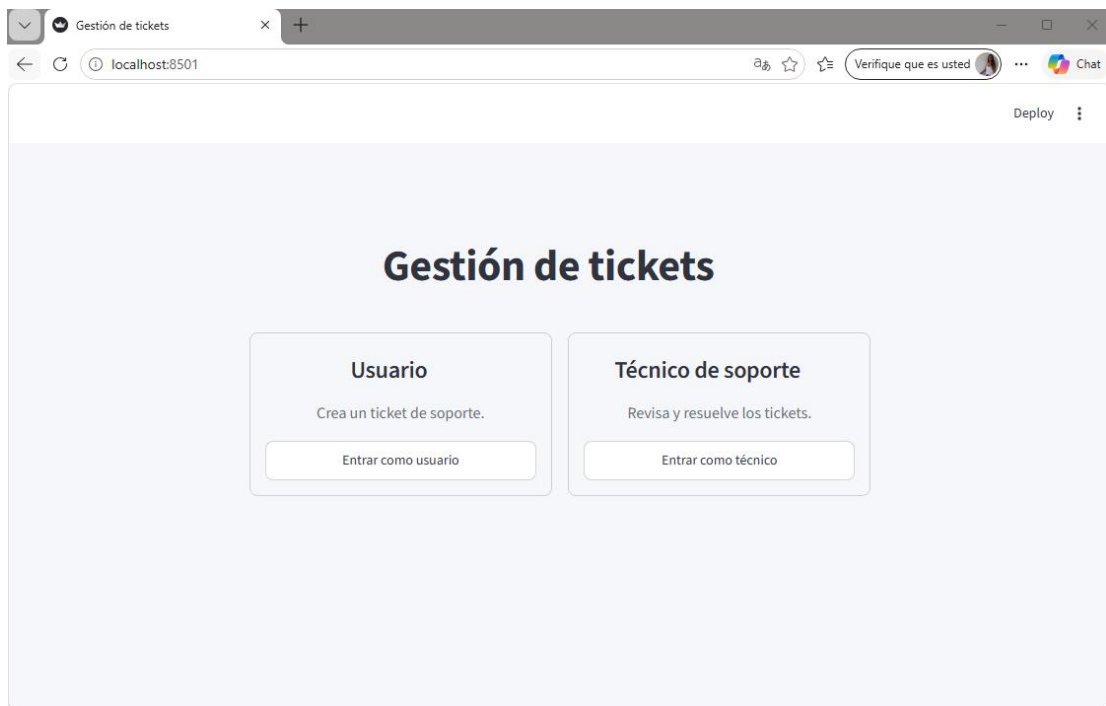


Figura A3.1

Entrar como usuario

Figura A3.1

Entrar como técnico: tickets a resolver

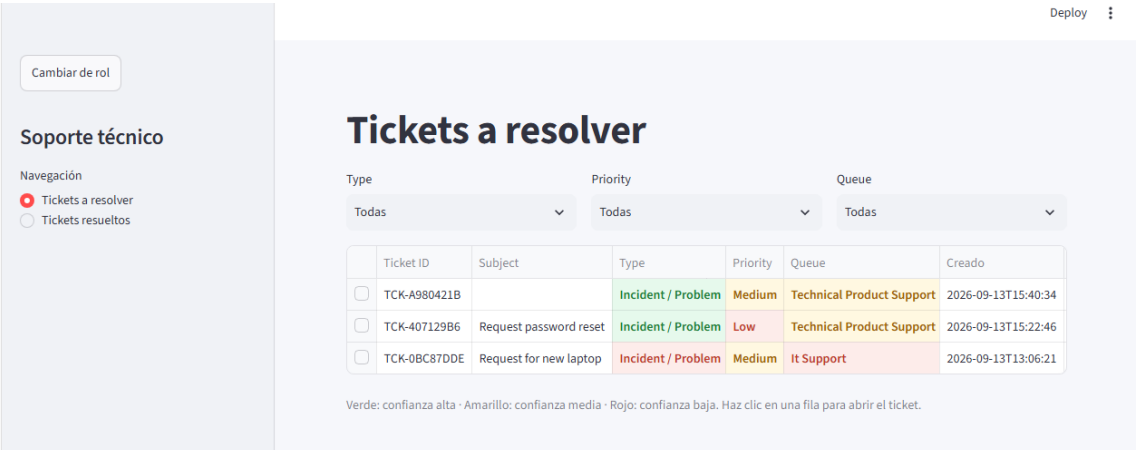


Figura A3.3

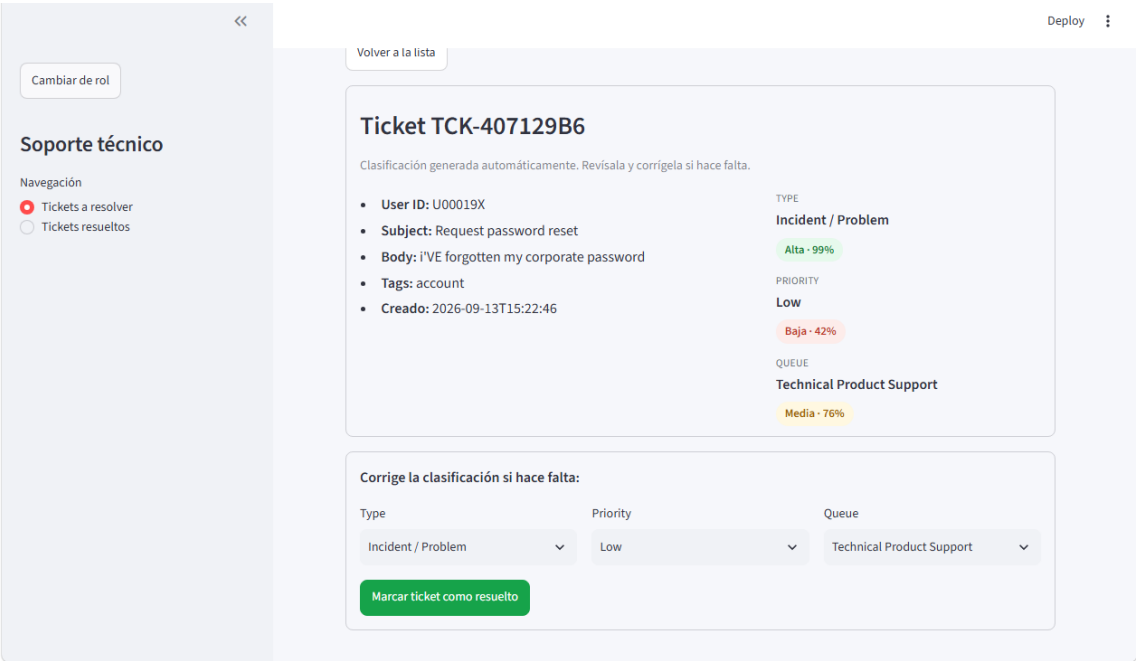


Figura A3.4

Entrar como técnico: tickets resueltos

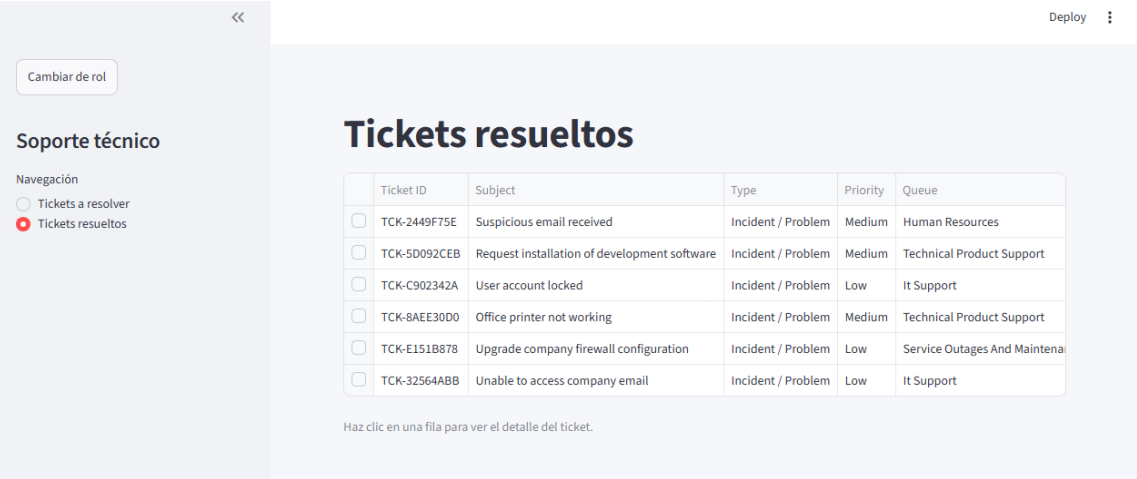


Figura A3.5

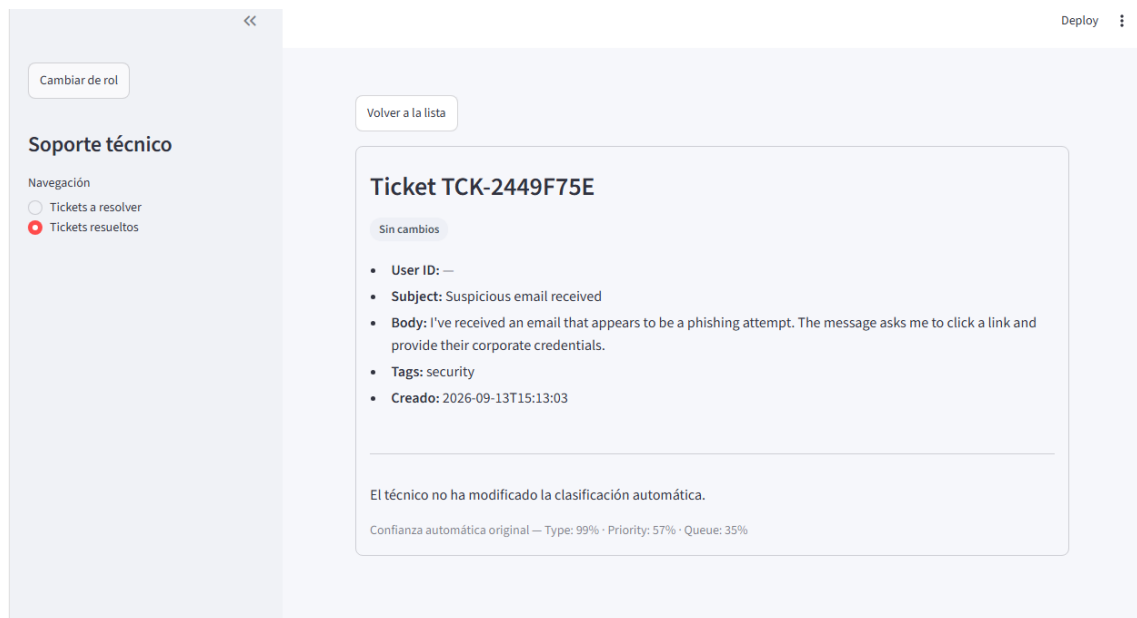


Figura A3.6